

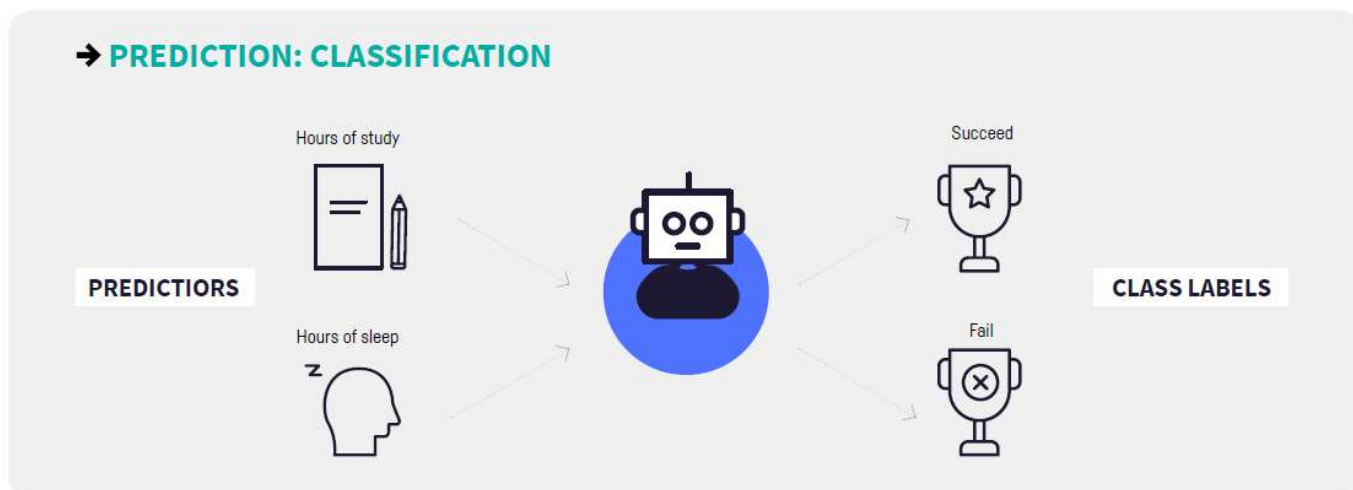
7. Overview and concepts

Next  >



Classification

Unlike regression, classification refers to a prediction task where the value to be predicted is nominal or categorical. For instance, we may want to predict whether students "succeed" or "fail" (categorical values) based on other data that we know (e.g. hours of study, hours of sleep), illustrated below:



Source: dataiku.com. (<https://pages.dataiku.com/machine-learning-basics-illustrated-guidebook>)

Naive Bayes algorithm

The Naive Bayes classifier is one of the simplest but potentially powerful classifiers. It is based on the Bayes Rule.

The Bayes Rule

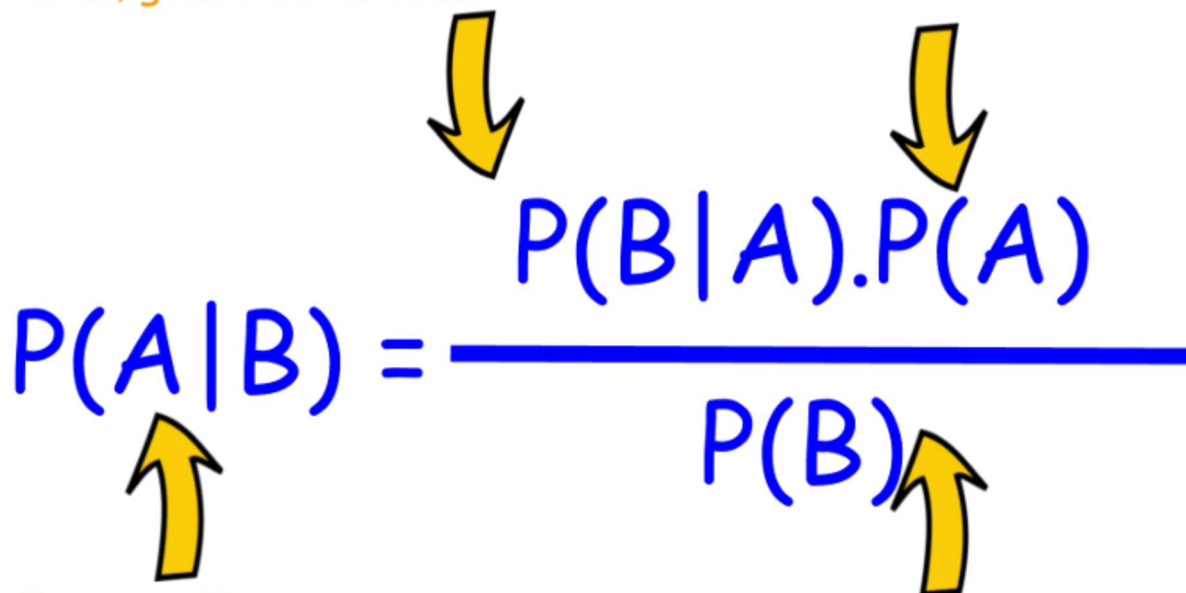
The Bayes Rule (the Bayes theorem or the Bayes formula) is one of the key principles in the theory of probability. It is particularly useful for calculating the probability of events marked by many uncertainties. The rule is a derivation of the conditional probability formula and has the following anatomy:

LIKELIHOOD

The probability of "B" being True, given "A" is True

PRIOR

The probability "A" being True. This is the knowledge.



The diagram illustrates the components of the Bayes Rule formula. At the top, 'LIKELIHOOD' and 'PRIOR' are defined with arrows pointing to the numerator of the formula. At the bottom, 'POSTERIOR' and 'MARGINALIZATION' are defined with arrows pointing to the denominator and the overall formula respectively. The formula itself is written in blue text.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

POSTERIOR

The probability of "A" being True, given "B" is True

MARGINALIZATION

The probability "B" being True.

Source: [towardsdatascience.com \(https://towardsdatascience.com/bayes-rule-with-a-simple-and-practical-example-2bce3d0f4ad0\)](https://towardsdatascience.com/bayes-rule-with-a-simple-and-practical-example-2bce3d0f4ad0)

Refer to the section **Bayes Rule derivation** of the learning materials this week if you are interested to know how we derive the Bayes Rule from the Conditional Probability equation.

Naive Bayes classifier

Unlike the Linear Regression algorithm which predicts numerical output values, Naive Bayes is typically used for predicting nominal/categorical output values. These nominal or categorical values are often called '*class*', such that the process of predicting them is termed **classification**.

Assuming that a dataset is in a tabular format with rows and columns, the Naive Bayes algorithm essentially works in two steps, as follows:

Step 1: calculate the probability of a class c_j given the input values $\mathbf{X}$ from a row (posterior probability). This is the *probability estimation* step.

This step involves a generalisation of the Bayes Rule above. Let c_j represent a specific class (a categorical value of the response variable) and $\mathbf{X}$ represent a set of explanatory variables' values, we replace A and B in the previous Bayes Rule with c_j and X , respectively:

$$P(c_j | X) = \frac{P(X | c_j) \cdot P(c_j)}{P(X)}$$

Next, assume that the response variable has at least two classes (because it is not meaningful to perform classification on a response variable with only one categorical value), the Naive Bayes classifier must calculate the posterior probability of the first class c_1 given X , $P(c_1 | X)$, and the probability of the second class c_2 given X , $P(c_2 | X)$. In this calculation, X are the same *regardless of the class*.

Since we are only interested to know how the classes' posterior probabilities compare relative to each other (i.e. which has the higher or lower probability score), it is safe to remove the common denominator $P(X)$ from the Bayesian equation to simplify and speed up its calculation:

$$P(c_j | X) = P(X | c_j) \cdot P(c_j)$$

Also, because X is made up of a set of explanatory variables' values that occur together (i.e. joint probability), $X=(x_1, x_2, x_3, \dots, x_i)$, we can further expand the formula:

$$\begin{aligned} P(c_j | X) &= P(x_1, x_2, x_3, \dots, x_i | c_j) \cdot P(c_j) \\ &= P(x_1 | c_j) \cdot P(x_2 | c_j) \cdot P(x_3 | c_j) \cdots P(x_i | c_j) \cdot P(c_j) \end{aligned}$$

The expansion above is possible given the naive assumption that the variables in X are independent of each other (see [probability of independent events \(https://brilliant.org/wiki/probability-independent-events/\)](https://brilliant.org/wiki/probability-independent-events/)).

Finding the posterior probability of a class is equivalent to asking the question "*What is the probability that a particular object belongs to a class given its observed feature values?*". For example, we may ask "What is the probability that a person has diabetes (c_1) or no diabetes (c_2) given a certain value for a pre-breakfast blood glucose measurement (x_1) and a certain value for a post-breakfast blood glucose measurement (x_2)?"

Step 2: once all posterior probabilities are calculated, the last step is to select a class with the highest probability score as the true class for each row in the data. This is the *classification* step.

Here, we show how to implement a Naive Bayes classifier above manually by constructing conditional probability tables in Microsoft Excel. These tables embody the equation above. This activity should give a better understanding of the workings of this classifier.

Example

The golf weather dataset consists of weather-related categorical explanatory variables for predicting whether to **play** or **not to play** golf. Construct the probability tables based on the following training set:



weather_train.xlsx

Using the conditional probability tables, perform classification on this test set:



weather_test.xlsx

Solution



weather_NB_solution.xlsx

Laplace Smoothing

The above solution also illustrates a problem: if any test case incorporates a zero probability into the Naive Bayes equation, it cancels the other non-zero probabilities and nullifies the other relevant input values that should have been counted in the classification.

To solve this "zero probability problem", in the solution, we added 1 to every frequency count in the conditional probability tables.

This clever trick is called the **Laplace Smoothing** (or Additive Smoothing), invented by the French scholar Pierre-Simon Laplace. Notice how the classification accuracy improves following the Laplace Smoothing.

Naive Bayes variants

The Naive Bayes algorithm is a *generative* model that performs classification by assuming that the explanatory variables' values are generated from a certain type of probability distribution. There are a few variants of the algorithm depending on the probability distribution it assumes. Knowing them is important because different variants are suited for different types of explanatory variables we deal with.

Categorical Naive Bayes

This is the same variant as the example above. It is suitable for scenarios where all explanatory variables are non-binary categorical (discrete) variables. It assumes that data are generated from the categorical distribution (<https://www.statology.org/categorical-distribution/>).

Bernoulli Naive Bayes

This variant assumes that data are drawn from the Bernoulli distribution (<https://brilliant.org/wiki/bernoulli-distribution/>) and that each explanatory variable has only two possible outcomes or categorical values (binary). Bernoulli distribution models the following situations:

- A newborn child is either male or female. (Here the probability of a child being a male is roughly 0.5.)
- A student either passes or fails an exam.
- A tennis player either wins or loses a match.
- A dart is thrown at a circular dartboard and land randomly over its area. The dart will either land closer to the centre than to the edge or not (in the second case it is either closer to the edge or equally distant from the centre and the edge).

Multinomial Naive Bayes

The multinomial Naive Bayes applies when it is believed that the explanatory variables come from the multinomial distribution (<https://stattrek.com/probability-distributions/multinomial.aspx>). This variant is suitable when each explanatory variable comprises more than two categorical values. Example: data from rolling a die 10 times to see what numbers we obtain.

The categorical distribution is a *special case* of the multinomial distribution with only a single trial. In some fields such as natural language processing, categorical and multinomial distributions are synonymous and it is common to speak of a multinomial distribution when referring to a categorical distribution.

Gaussian Naive Bayes

The Gaussian Naive Bayes should be used when all explanatory variables are made up of continuous values. It assumes these values arise from the Gaussian distribution (<https://www.mathsisfun.com/data/standard-normal-distribution.html>) (i.e. Normal distribution). The implied limitation of the model is that it may not work well if the values follow a non-Gaussian distribution, are skewed, or contain significant outliers.

Applications

The Naive Bayes algorithm is extremely fast and scalable against large datasets. It is well known to work well in such applications as email spam detection and filtering, natural language processing, and recommendation systems.

Assumptions and limitations

There are, however, limitations to the algorithm. It makes a very strong assumption about the *conditional independence* among the input variables X . It believes that the values of one input variable do not affect the values of all other input variables in a dataset (hence "Naive" in that sense), represented by this component of the equation:

$$P(x_1 | c_j) \cdot P(x_2 | c_j) \cdot P(x_3 | c_j) \cdots P(x_i | c_j)$$

Unfortunately, much of our reality does not conform with this assumption such that there are chances that the Naive Bayes algorithm will not produce accurate predictions. Also, owing to this naive assumption, it is common in practice to use the Naive Bayes algorithm only as a benchmark baseline to compare against other better classifiers.

Ironically, the same limitation above gives the algorithm its phenomenal speed. Also, even though its conditional independence assumption is often not realistic and often leads to poor probability estimation (see **Step 1** of the Naive Bayes algorithm explained above), the difference between the probability score of the true class versus that of the false class is often large enough to ensure the algorithm is capable of picking the right class (which means **Step 2** still works well). This explains why the algorithm still performs well, particularly for natural language processing tasks (such as text sentiment classification). However, the probability of certain words appearing in natural language texts is usually not conditionally independent from the presence of other words.